

Software Architecture & Design Specification

Student Record System

1. Introduction

1.1 Purpose

This document specifies the software architecture and detailed design of the Student Record System (SRS-App), a menu-driven C/C++ application for maintaining student details, grades and attendance..

1.2 Scope

The specification covers the system's architectural style, component decomposition, requirements traceability, security architecture, interface (API) design at the module level, error handling strategy, and representative dynamic behaviour through UML sequence diagrams. It applies to the core scope defined in the SRS: student CRUD, grades, attendance, search/filter, validation and persistent storage.

1.3 Definitions, Acronyms and Abbreviations

- FR / NFR / MTR - Functional / Non-Functional / Technical Requirement
- UC / AF - Use Case / Alternative Flow
- UT / FT / ST / AT - Unit / Functional / System / Acceptance Test
- API - Application Programming Interface (here: internal module-level function interfaces, not a network API)

1.4 References

- Student Record System - Software Requirements Specification & Requirements Traceability Matrix (
- Student Record System - Test Plan Document

2. Architecture

2.1 Architectural Pattern

The system follows a Layered (N-tier) Architecture, chosen for its fit with a single-user, menu-driven C/C++ console application and its natural separation of concerns:

- Presentation Layer - main.cpp: role-based main menu, console input/output, and the exit function (FR-04, FR-25, NFR-01, NFR-10).
- Application Layer - student.cpp, grades.cpp, attendance.cpp, search.cpp, report.cpp, auth.cpp: implements the business logic for each functional area.
- Validation Layer - validation.cpp: a cross-cutting layer invoked by the Application layer to check input before it is acted upon or persisted.
- Data Layer - storage.cpp (with backup.cpp and student.h): responsible for persistent storage (file-based: students.dat / students.csv), backup/recovery and the data model.

Within the Application layer, related functions are further grouped as modular components (one .cpp/.h pair per functional area), giving the system characteristics of a lightweight component-based

architecture layered on top of the four tiers above. This combination directly satisfies MTR-01 (modular C/C++ architecture) and MTR-04 (separation of concerns) and supports NFR-05 (modular design) and NFR-09 (portability, since each module depends only on standard C/C++ and the layer below it).

2.2 Component Diagram and Description

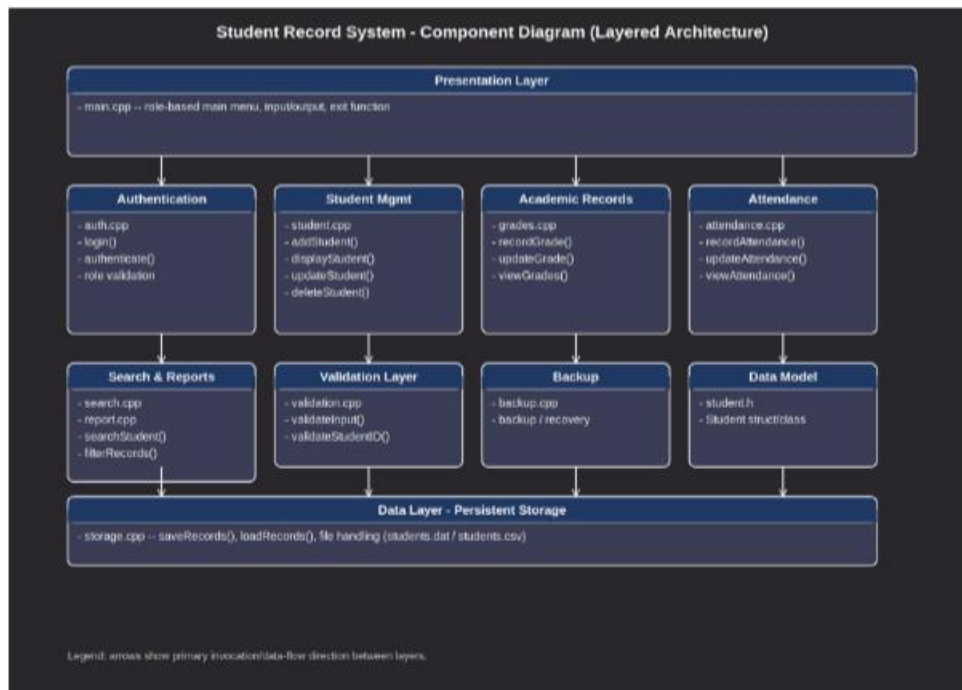


Figure 1. Component diagram showing the four architectural layers and their module-level components.

Component responsibilities:

- Authentication (auth.cpp): login(), authenticate(), and role validation; the sole entry point for establishing an actor's identity and role (Administrator, Faculty, Student).
- Student Management (student.cpp): addStudent(), displayStudent(), updateStudent(), deleteStudent() - core CRUD operations on student records.
- Academic Records (grades.cpp): recordGrade(), updateGrade(), viewGrades().
- Attendance (attendance.cpp): recordAttendance(), updateAttendance(), viewAttendance().
- Search & Reports (search.cpp, report.cpp): searchStudent(), filterRecords(), displayResults().
- Validation Layer (validation.cpp): validateInput(), validateStudentID() - shared by every component that accepts actor input.
- Backup (backup.cpp): periodic/on-demand backup and recovery of persisted data (NFR-06).
- Data Model (student.h): the Student struct/class and related record types shared across components.
- Data Layer (storage.cpp): saveRecords(), loadRecords() and file handling against students.dat / students.csv - the only component that performs file I-O.

Dependencies flow strictly downward: Presentation depends on Application; Application depends on Validation and the Data Model; Validation, Backup and the Data Model depend on the Data Layer. No component in a lower layer calls back into a higher layer, which keeps the design testable in isolation (supporting the Unit Test level in the Test Plan).

2.3 Traceability to Requirements

The table below summarizes how architectural elements trace to the requirements defined in the SRS and Requirements Traceability Matrix. Full per-requirement Test Case IDs are maintained in that matrix; this table groups related requirements by architectural concern.

Req. ID(s)	Description	Layer / Concern	Realizing Module(s)	Test ID Range
FR-01/02/03/04	Authentication & role-based access	Application/Presentation	auth.cpp (login, authenticate, role validation); main.cpp (menu)	UT/FT/ST/AT-01..04
FR-05..11	Student CRUD (add/view/search/update/delete)	Application	student.cpp, search.cpp	UT/FT/ST/AT-05..11
FR-12..14	Grade record/update/view	Application	grades.cpp	UT/FT/ST/AT-12..14
FR-15..17	Attendance record/update/view	Application	attendance.cpp	UT/FT/ST/AT-15..17
FR-18/19	Filter & reporting	Application	report.cpp	UT/FT/ST/AT-18..19
FR-20/21	Input validation & handling	Validation	validation.cpp	UT/FT/ST/AT-20..21
FR-22..24	Persistent storage (save/load/file I-O)	Data	storage.cpp	UT/FT/ST/AT-22..24
FR-25	Safe exit	Presentation	main.cpp	UT/FT/ST/AT-25
NFR-01/10	Usability & UI consistency	Presentation	main.cpp	UT/FT/ST/AT-01,10 (NFR)
NFR-02/11	Performance on up to 1,000 records	Application/Data	search.cpp, storage.cpp	UT/FT/ST/AT-02,11 (NFR)

Req. ID(s)	Description	Layer / Concern	Realizing Module(s)	Test ID Range
NFR-03/07	Data integrity & reliability	Data	student.cpp, storage.cpp	UT/FT/ST/AT-03,07 (NFR)
NFR-04	Error handling	Validation/Presentation	validation.cpp, main.cpp	UT/FT/ST/AT-04 (NFR)
NFR-05/09	Modular design & portability	System Architecture	module .h/.cpp files	UT/FT/ST/AT-05,09 (NFR)
NFR-06	Backup & recovery	Data	backup.cpp, storage.cpp	UT/FT/ST/AT-06 (NFR)
NFR-08/12	Access control & data protection	Application/Data	auth.cpp, student.cpp, storage.cpp	UT/FT/ST/AT-08,12 (NFR)
MTR-01..04	C/C++ modular architecture, storage & data model design	System Architecture/Data	main.cpp + module files, storage.cpp, student.h	UT/FT/ST/AT (MTR)

2.4 Security Architecture

Security is addressed primarily through role-based access control, input validation and data protection, consistent with NFR-08 and NFR-12:

- Authentication boundary: auth.cpp is the single point where credentials are checked (FR-01, FR-02) before any other component is reachable from the Presentation layer.
- Role-based access control: after authentication, auth.cpp's role validation determines which menu options main.cpp exposes (FR-03, FR-04); Application-layer components do not re-implement authorization logic, avoiding inconsistent checks.
- Least privilege by role: Administrator has no access to grade or attendance modification; Faculty has no access to student deletion; Student is restricted to read-only views of their own data (own profile, own grades, own attendance), enforced at the menu (Presentation) layer and re-validated at the Application layer for defence in depth.
- Input validation as a security control: validation.cpp defends the Application and Data layers against malformed or malicious input (NFR-04), reducing the risk of corrupted records or logic errors in storage.cpp.
- Data protection: storage.cpp restricts write operations to authorized flows only (NFR-08); backup.cpp provides recovery if a write is interrupted or a file is corrupted (NFR-06, NFR-07).
- Threat notes and limitations: as a single-user local console application, the system does not implement network-facing security controls (e.g. encryption in transit); the primary threats addressed are unauthorized in-application actions and malformed input, not remote attack. If

persisted files are stored in a shared filesystem, OS-level file permissions should be relied upon for at-rest protection.

2.5 Other Architectural Sections

2.5.1 Deployment View

The system is deployed as a single standalone executable on a local machine; there is no client-server split. All modules compile into one binary; students.dat / students.csv reside alongside the executable or in a configured data directory.

2.5.2 Data View

The primary persisted entity is the Student record (student.h), which references associated Grade and Attendance records. Data is stored in a flat file format (binary .dat or delimited .csv) as chosen in MTR-02, loaded into memory on demand by loadRecords() and written back by saveRecords().

2.5.3 Quality Attributes Summary

- Performance (NFR-02, NFR-11): search.cpp and storage.cpp are designed to complete common operations within 2 seconds for up to 1,000 records via linear or indexed in-memory search after a single load.
- Reliability (NFR-07) and Backup (NFR-06): storage.cpp and backup.cpp cooperate to avoid data loss on interrupted writes.
- Portability (NFR-09): standard C/C++ only, no platform-specific APIs, enabling compilation across common environments.

3. Design

3.1 UML Sequence Diagrams

Two representative sequence diagrams are provided below, each showing a primary success flow together with its alternative (error) flow as a UML alt combined fragment, consistent with the Alternative Flow Scenarios in the SRS.

3.1.1 Sequence Diagram 1 - UC-01: Login

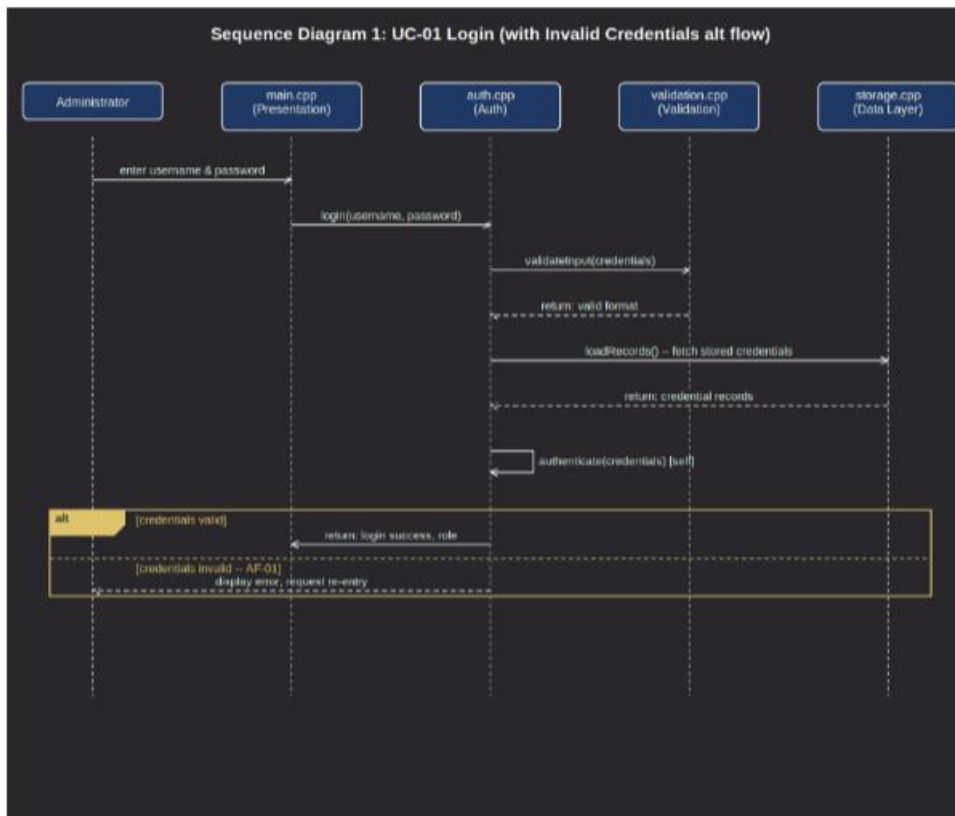


Figure 2. Login sequence, including the AF-01 Invalid Credentials alternative flow.

This flow shows main.cpp collecting credentials from the actor and delegating to auth.cpp. auth.cpp first calls validation.cpp to check input format, then loadRecords() from storage.cpp to retrieve stored credential records, and finally performs authenticate() as a self-call before returning success (with role) or triggering AF-01 to display an error and request re-entry.

3.1.2 Sequence Diagram 2 - UC-06: Record Grades

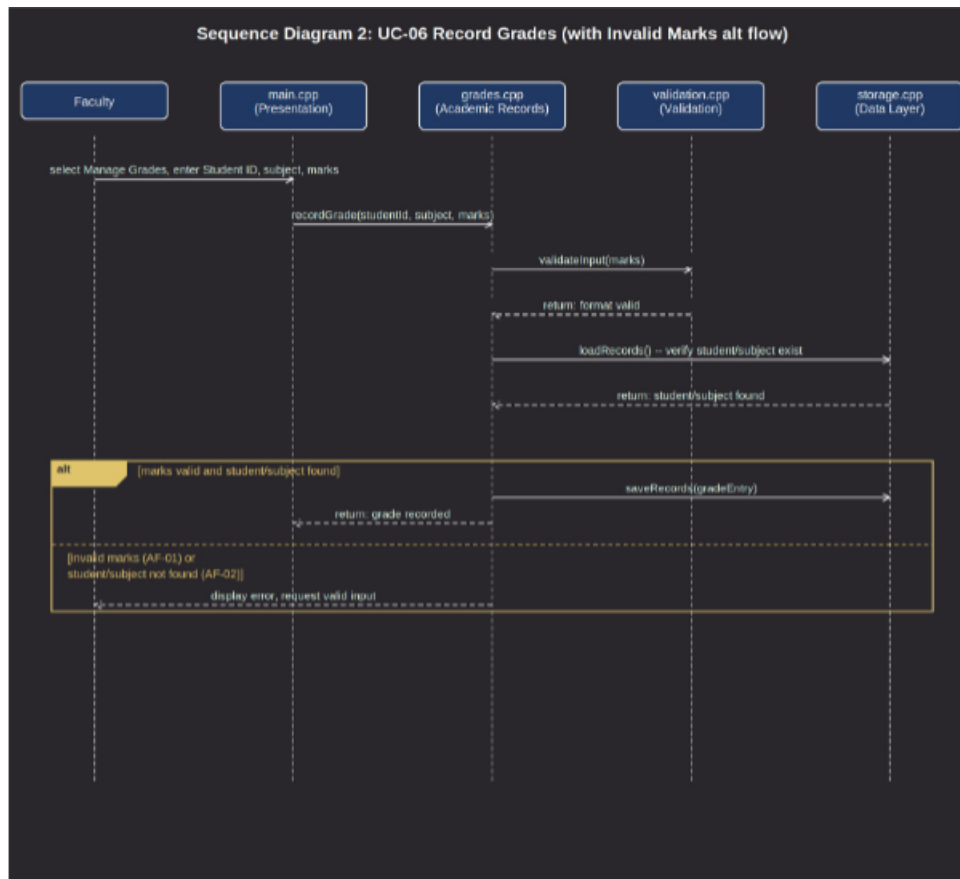


Figure 3. Record Grades sequence, including the AF-01/AF-02 Invalid Marks / Student-or-Subject-Not-Found alternative flow.

This flow shows Faculty invoking Manage Grades via main.cpp, which calls grades.cpp's recordGrade(). Marks are validated by validation.cpp and existence of the student/subject is checked via storage.cpp before the grade entry is persisted with saveRecords(); otherwise the AF-01/AF-02 branch displays an error and requests valid input without writing to storage.

3.2 API Design (Module-Level Interfaces)

As a standalone C/C++ application, the system exposes no network API. 'API' here refers to the internal function-level interfaces each module provides to its callers, which are treated as contracts for integration and unit testing.

Function(s)	Module	Signature (conceptual)
login()	auth.cpp	username: string, password: string -> returns: AuthResult{success, role, message}
authenticate()	auth.cpp	credentials: Credentials, stored: CredentialRecord[] -> returns: bool
addStudent()	student.cpp	student: StudentRecord -> returns: OperationResult{success, message}

Function(s)	Module	Signature (conceptual)
displayStudent()/displayAllStudents()	student.cpp	studentId: string (optional) -> returns: StudentRecord or StudentRecord[]
updateStudent()	student.cpp	studentId: string, updated: StudentRecord -> returns: OperationResult
deleteStudent()	student.cpp	studentId: string, confirm: bool -> returns: OperationResult
searchStudent()	search.cpp	query: string (ID or name) -> returns: StudentRecord[] (0..n matches)
recordGrade()/updateGrade()	grades.cpp	studentId: string, subject: string, marks: int -> returns: OperationResult
viewGrades()	grades.cpp	studentId: string -> returns: GradeRecord[]
recordAttendance()/updateAttendance()	attendance.cpp	studentId: string, date: string, status: enum -> returns: OperationResult
viewAttendance()	attendance.cpp	studentId: string -> returns: AttendanceRecord[]
filterRecords()	report.cpp	criteria: FilterCriteria{course, department, semester} -> returns: StudentRecord[]
validateInput()/validateStudentID()	validation.cpp	raw: string, type: FieldType -> returns: ValidationResult{valid, message}
saveRecords()/loadRecords()	storage.cpp	records: T[], filePath: string -> returns: bool / T[] (file I-O on students.dat or students.csv)

Conventions: every mutating function (add/update/delete/record) returns an OperationResult carrying a success flag and a human-readable message rather than throwing exceptions, so main.cpp can present a consistent message to the actor regardless of which module raised it. Every read function returns either a populated collection/record or an explicitly empty result, which callers treat as a valid 'no data' outcome rather than an error.

3.3 Error Handling

Error handling follows a consistent pattern across modules: validate as early as possible, fail without side effects, and report a clear, actionable message back through the Presentation layer. The table below maps error categories to the responsible module and the SRS alternative flow they implement.

Error Category	Related Use Case / AF	Handling Strategy
Invalid credentials / empty fields	Login (UC-01, AF-01/AF-02)	Reject attempt, display specific error message, return control to login prompt for re-entry; no partial session created.
Duplicate Student ID / invalid details	Add Student (UC-02, AF-01/AF-02)	Reject the write before it reaches storage.cpp; display message; discard partial input; return to Add Student menu.
Student not found	Search/Update/Delete/Grades/Attendance (multiple AFs)	validateStudentID() or a storage lookup fails to match; system displays a not-found message and returns to the calling menu without altering data.
Invalid marks / attendance value	Manage Grades / Attendance (AF-01)	validateInput() rejects out-of-range or malformed values before any call to storage.cpp; user is re-prompted.
User cancels operation	Add/Update/Delete (various AF)	In-memory changes are discarded; no write is issued to storage.cpp; prior persisted state is untouched.
No matching filter/search results	Filter/Search (AF-01)	Empty result set is treated as a valid, non-error outcome; a clear 'no records found' message is shown.
File I-O failure (storage.cpp)	Cross-cutting (NFR-06, NFR-07)	Operation aborts, error is logged, backup.cpp is invoked where applicable to restore the last known-good state; user is informed the action did not complete.
Unauthorized access attempt	Cross-cutting (NFR-08, NFR-12)	Role validation in auth.cpp denies the operation before it reaches the Application layer; attempt is not executed or persisted.

General principles: (1) validation.cpp is the single point for input-format checks, avoiding duplicated or inconsistent validation logic across modules; (2) no partial writes reach storage.cpp - a record is either fully validated and saved, or the operation is aborted and the previous persisted state is unchanged; (3) all user-facing error messages are generated by the module that detected the error and are passed upward unchanged, so the meaning is not lost in translation between layers; (4) file-level failures are treated as system-level errors distinct from input-validation errors, and are logged and, where possible, recovered via backup.cpp.

3.4 Other Design Sections

3.4.1 Data Structures

The core Student data model (student.h) holds Student ID, Name, Contact Details, Course/Department and Semester, together with references to associated Grade records (Subject, Marks) and Attendance records (Date, Status), as defined in the SRS's 'Main Data Maintained' section.

3.4.2 Design Rationale

A layered architecture with one module per functional area was chosen over a monolithic main.cpp to keep the two-month implementation timeline manageable: each module can be implemented, unit-tested (per the Test Plan's UT-xx cases) and reviewed independently, and the strict downward dependency rule limits the blast radius of any single defect.

3.4.3 Traceability Back to Test Plan

Each API function listed in 3.2 and each error-handling behaviour in 3.3 corresponds to at least one test case already defined in the Test Plan Document (e.g. TC-FR-01/01b/01c for login, TC-FR-12 for grade validation, TC-FR-11 for delete-cancel), so this design can be validated directly against existing test cases without additional planning.